COMPONENTE CURRICULAR:	Programação de Sistemas II
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

1. **Interface Tributavel.java: "Define o contrato para cálculo de impostos."**

Tributavel.java U

MinhaCarteira > Tributavel.java > Tributavel

```java
public interface Tributavel {
    // Todo objeto tributável deve obrigatoriamente calcular seu imposto
    double calcularImposto();
}
```

2. **Classe Abstrata Transacao.java: "Classe base que estabelece os atributos comuns e o método abstrato exibirDetalhes()."**

Transacao.java U

MinhaCarteira > Transacao.java > Transacao

```java
public abstract class Transacao {
    private String descricao;
    private double valor;
    private String data;

    // Construtor para inicializar os atributos
    public Transacao(String descricao, double valor, String data) {
        this.descricao = descricao;
        this.valor = valor;
        this.data = data;
    }

    // Getters e Setters (Encapsulamento)
    public String getDescricao() { return descricao; }
    public void setDescricao(String descricao) { this.descricao = descricao; }

    public double getValor() { return valor; }
    public void setValor(double valor) { this.valor = valor; }

    public String getData() { return data; }
    public void setData(String data) { this.data = data; }

    // Método que as classes filhas serão obrigadas a implementar
    public abstract void exibirDetalhes();
}
```

3. **Classe Receita.java: "Implementação de entrada de capital, herdando de Transacao e aplicando a interface Tributavel com a regra de 10%."**

Receita.java U

MinhaCarteira > Receita.java > ...

```java
public class Receita extends Transacao implements Tributavel {

    public Receita(String descricao, double valor, String data) {
        super(descricao, valor, data);
    }

    @Override
    public double calcularImposto() {
        // Regra de negócio: 10% do valor
        return getValor() * 0.10;
    }

    @Override
    public void exibirDetalhes() {
        System.out.println("+++ DETALHE DE RECEITA +++");
        System.out.println("Data: " + getData());
        System.out.println("Descrição: " + getDescricao());
        System.out.println("Valor: R$ " + getValor());
        System.out.println("Imposto Estimado (10%): R$ " + calcularImposto());
        System.out.println("++++++++++++++++++++++++++++");
    }
}
```

4. **Classe Despesa.java: "Implementação de saída de capital, herdando de Transacao."**

Despesa.java U

MinhaCarteira > Despesa.java > Despesa

```java
public class Despesa extends Transacao {

    public Despesa(String descricao, double valor, String data) {
        super(descricao, valor, data);
    }

    @Override
    public void exibirDetalhes() {
        System.out.println("--- DETALHE DE DESPESA ---");
        System.out.println("Data: " + getData());
        System.out.println("Descrição: " + getDescricao());
        System.out.println("Valor: -R$ " + getValor());
        System.out.println("--------------------------");
    }
}
```

5. **Classe Sistema.java: "Classe principal que demonstra o uso de polimorfismo através de uma List<Transacao> e o uso do operador instanceof."**

```java
Sistema.java U ×
MinhaCarteira > Sistema.java > Sistema
import java.util.ArrayList;
import java.util.List;

public class Sistema {
    Run | Debug
    public static void main(String[] args) {
        // Lista polimórfica: aceita qualquer objeto que seja uma Transacao
        List<Transacao> minhasTransacoes = new ArrayList<>();

        // Instanciando os objetos conforme solicitado
        minhasTransacoes.add(new Despesa(descricao: "Conta de Luz", valor: 150.00, data: "10/02/2026"));
        minhasTransacoes.add(new Receita(descricao: "Salário Mensal", valor: 5000.00, data: "05/02/2026"));
        minhasTransacoes.add(new Despesa(descricao: "Supermercado", valor: 600.00, data: "12/02/2026"));

        // Loop para percorrer a lista
        for (Transacao t : minhasTransacoes) {
            t.exibirDetalhes(); // Polimorfismo: chama o método da classe correta

            // Verificação de instância para imposto
            if (t instanceof Tributavel) {
                // Cast (conversão temporária) para acessar o método da interface
                Tributavel objTributavel = (Tributavel) t;
                System.out.println("LOG: Processando tributação de " + t.getDescricao() +
                                   " | Imposto: R$ " + objTributavel.calcularImposto());
            }
            System.out.println(); // Pular linha para organizar
        }
    }
}
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    Run: Sistema

PS C:\Users\murillo.silva\Documents\GitHub\ads\semestre-3\programacao-de-sistemas-ii\A1-aplicando-connhecimento>  & 'C:\Program Files\Java\jre1.8.0_421\bin\java
.exe' '-cp' 'C:\Users\murillo.silva\AppData\Roaming\Code\User\workspaceStorage\3679fc6512a22a03fcbc9612613de79c\redhat.java\jdt_ws\A1-aplicando-connhecimento_74
9c4d8b\bin' 'Sistema'
--- DETALHE DE DESPESA ---
Data: 10/02/2026
Descrição: Conta de Luz
Valor: -R$ 150.0
-------------------------

+++ DETALHE DE RECEITA +++
Data: 05/02/2026
Descrição: Salário Mensal
Valor: R$ 5000.0
Imposto Estimado (10%): R$ 500.0
++++++++++++++++++++++++++
LOG: Processando tributação de Salário Mensal | Imposto: R$ 500.0

--- DETALHE DE DESPESA ---
Data: 12/02/2026
Descrição: Supermercado
Valor: -R$ 600.0
-------------------------

PS C:\Users\murillo.silva\Documents\GitHub\ads\semestre-3\programacao-de-sistemas-ii\A1-aplicando-connhecimento>
```